Abstraction in C# - Comprehensive Study Notes

Overview: What is Abstraction?

Abstraction is a fundamental principle of object-oriented programming that focuses on **hiding implementation details** while exposing only essential features to the user. It allows developers to work with complex systems by focusing on **what an object does** rather than **how it does it**.[1][2][3][4]

Key Purpose:

- Simplify complex systems by showing only relevant operations
- Hide internal implementation complexity
- Create frameworks for other classes to build upon
- Improve code maintainability and scalability[4][5]

Two Ways to Achieve Abstraction in C#:

1. **Abstract Classes** (0-100% abstraction)
2. **Interfaces** (100% abstraction - until C# 8.0)

Abstract Classes in C#

Definition and Characteristics

An **abstract class** is a class that **cannot be instantiated directly**. It serves as a blueprint for other classes and can contain both abstract members (without implementation) and concrete members (with implementation).[6][7][3][1]

Key Features:

- Cannot create objects directly from abstract class
- Can contain abstract methods (no body) and non-abstract methods (with body)
- Can have fields, properties, constructors, destructors
- Can have static members and constants
- Supports all access modifiers (public, protected, private, internal)[8][7][1]

Members Allowed in Abstract Classes

Member Type	Allowed	Can Have Implementation
Fields (variables)	☑ Yes	☑ Yes
Constants	☑ Yes	☑ Yes
Constructors	☑ Yes	☑ Yes
Destructors	☑ Yes	☑ Yes
Abstract methods	☑ Yes	❌ No (declaration only)
Concrete methods	☑ Yes	☑ Yes
Virtual methods	☑ Yes	☑ Yes
Properties	☑ Yes	☑ Yes (abstract or concrete)
Events	☑ Yes	☑ Yes
Indexers	☑ Yes	☑ Yes
Static members	☑ Yes	☑ Yes

Abstract Class Example

```
abstract class Vehicle
{
    // Fields (instance variables)
    protected string _brand;
    private int _year;

    // Constants
    public const int MaxSpeed = 200;

    // Constructor
    public Vehicle(string brand, int year)
    {
        _brand = brand;
        _year = year;
        Console.WriteLine("Vehicle constructor called");
    }
```

```
    // Concrete method with implementation
    public string GetInfo()
    {
        return $"This is a {_brand} vehicle from {_year}";
    }

    // Virtual method (can be overridden)
    public virtual void StartEngine()
    {
        Console.WriteLine($"{_brand} engine is starting...");
    }

    // Abstract method (must be implemented by derived classes)
    public abstract void Move();

    // Abstract property (must be implemented)
    public abstract int CurrentSpeed { get; set; }

    // Concrete property
    public int Year
    {
        get { return _year; }
        set { _year = value; }
    }
}

class Car : Vehicle
{
    private int _speed;

    public Car(string brand, int year) : base(brand, year) { }

    // Implementation of abstract method
    public override void Move()
    {
        Console.WriteLine($"{_brand} car is driving on the road");
    }

    // Implementation of abstract property
    public override int CurrentSpeed
```

```
    {
        get { return _speed; }
        set { _speed = value; }
    }

    // Override virtual method
    public override void StartEngine()
    {
        base.StartEngine(); // Call base implementation
        Console.WriteLine("Car engine started!");
    }
}

// Usage
Car car = new Car("Toyota", 2024);
car.StartEngine();
car.Move();
car.CurrentSpeed = 60;
```

Abstract Class Constructors

Important: Abstract classes **CAN have constructors**, even though they cannot be instantiated directly.[9][10][6]

Purpose of Constructors in Abstract Classes:

- Initialize fields of the abstract class
- Called when derived class objects are created
- Execute base class initialization logic

```
abstract class Animal
{
    protected string name;

    // Constructor in abstract class
    public Animal(string animalName)
    {
        name = animalName;
        Console.WriteLine($"Animal constructor: {name}");
```

```
    }

    public abstract void MakeSound();
}

class Dog : Animal
{
    public Dog(string name) : base(name) // Calls base constructor
    {
        Console.WriteLine("Dog constructor");
    }

    public override void MakeSound()
    {
        Console.WriteLine($"{name} says: Woof!");
    }
}

// Output when creating Dog:
// Animal constructor: Buddy
// Dog constructor
```

Abstract Properties

Abstract classes can have **abstract properties** that derived classes must implement.[7][11][6]

```
abstract class Shape
{
    public Shape(string s)
    {
        Id = s;
    }

    public string Id { get; set; }

    // Abstract property - read-only
    public abstract double Area { get; }

    // Abstract property - read-write
```

```
    public abstract string Color { get; set; }
}

class Circle : Shape
{
    private double _radius;

    public Circle(string id, double radius) : base(id)
    {
        _radius = radius;
    }

    public override double Area
    {
        get { return Math.PI * _radius * _radius; }
    }

    public override string Color { get; set; }
}
```

Interfaces in C#

Definition and Characteristics

An **interface** defines a **contract** that classes must follow. Traditionally, interfaces contained only declarations (no implementation), but **C# 8.0** introduced significant changes.[12][13][3][4]

Traditional Interface Features (Pre-C# 8.0):

- Only method, property, event, and indexer declarations
- No implementation allowed
- No fields or instance variables
- All members implicitly public
- Cannot contain constructors or destructors
- Classes can implement multiple interfaces[3][14][4]

C# 8.0+ Interface Features:

- **Default implementations** for methods and properties
- **Static members** allowed
- **Access modifiers** (public, private, protected, internal)
- **Virtual and abstract members**
- Interface evolution without breaking changes[15][13][16][12]

Members Allowed in Interfaces

Member Type	Pre-C# 8.0	C# 8.0+	Can Have Implementation
Fields (variables)	✗ No	✗ No	✗ No
Constants	☑ Yes	☑ Yes	☑ Yes
Methods (abstract)	☑ Yes	☑ Yes	✗ No (pre-8.0)
Methods (default)	✗ No	☑ Yes	☑ Yes (C# 8.0+)
Properties	☑ Yes (declaration)	☑ Yes	☑ Yes (C# 8.0+)
Events	☑ Yes	☑ Yes	☑ Yes (C# 8.0+)
Indexers	☑ Yes	☑ Yes	☑ Yes (C# 8.0+)
Static methods	✗ No	☑ Yes	☑ Yes (C# 8.0+)[17][18]
Constructors	✗ No	✗ No	✗ No

Traditional Interface Example (Pre-C# 8.0)

```
public interface ILogger
{
    // Method declaration only - no body
    void Log(string message);

    // Property declaration only
    string LogLevel { get; set; }

    // Event declaration
    event EventHandler LogCreated;
```

```
    // Indexer declaration
    string this[int index] { get; }
}

public class FileLogger : ILogger
{
    private string _logLevel;

    // Must implement all interface members
    public void Log(string message)
    {
        Console.WriteLine($"[{LogLevel}] {message}");
    }

    public string LogLevel
    {
        get { return _logLevel; }
        set { _logLevel = value; }
    }

    public event EventHandler LogCreated;

    public string this[int index]
    {
        get { return $"Log entry {index}"; }
    }
}
```

C# 8.0+ Interface Features: Default Interface Methods

Default Method Implementation

C# 8.0 introduced the ability to provide **default implementations** for interface methods, enabling interface evolution without breaking existing code.[13][19][15][12]

Key Benefits:

1. **Backward Compatibility**: Add new methods without breaking existing implementations
2. **Code Reusability**: Share common logic across implementations
3. **Interface Evolution**: Extend interfaces over time
4. **Versioning**: Update APIs without breaking changes[16][15]

Default Method Example

```
public interface IDevice
{
    // Abstract method - must be implemented
    void TurnOn();

    // Default method with implementation (C# 8.0+)
    void ShowInfo()
    {
        Console.WriteLine("This is a device");
    }

    // Default method calling another member
    void Initialize()
    {
        Console.WriteLine("Initializing device...");
        TurnOn();
    }
}

public class Phone : IDevice
{
    public void TurnOn()
    {
        Console.WriteLine("Phone is turning on");
    }

    // Can use default ShowInfo() or override it
}

public class Laptop : IDevice
{
    public void TurnOn()
```

```
    {
        Console.WriteLine("Laptop is turning on");
    }

    // Override default implementation
    public void ShowInfo()
    {
        Console.WriteLine("This is a laptop device");
    }
}

// Usage
IDevice phone = new Phone();
phone.TurnOn();        // Output: Phone is turning on
phone.ShowInfo();      // Output: This is a device (default)

IDevice laptop = new Laptop();
laptop.TurnOn();       // Output: Laptop is turning on
laptop.ShowInfo();     // Output: This is a laptop device (overridden)
```

Important Note: Default methods are only accessible when the object is **treated as the interface type**, not the concrete class type.[13]

```
IDevice device = new Phone();
device.ShowInfo();  // ☑ Works - accessed through interface

Phone phone = new Phone();
phone.ShowInfo();   // X Error - not accessible through class type
                    // unless explicitly implemented in class
```

Access Modifiers in Interface Methods (C# 8.0+)

C# 8.0+ allows **access modifiers** in interfaces to control member visibility.[20][21][12][13]

Supported Access Modifiers:

- **public** (default) - Accessible to all implementers
- **private** - Only accessible within the interface itself

- **protected** - Accessible in derived interfaces
- **internal** - Accessible within the same assembly
- **protected internal** - Union of protected and internal

```
public interface IAdvancedLogger
{
    // Public method (default) - must be implemented
    void Log(string message);

    // Public default method - available to implementers
    public void LogInfo(string message)
    {
        FormatLog("INFO", message);
    }

    // Private method - helper for other default methods
    private void FormatLog(string level, string message)
    {
        Console.WriteLine($"[{level}] {DateTime.Now}: {message}");
    }

    // Protected method - available in derived interfaces
    protected void LogInternal(string message)
    {
        FormatLog("INTERNAL", message);
    }

    // Virtual method - can be overridden
    public virtual void LogWarning(string message)
    {
        FormatLog("WARNING", message);
    }
}

class ConsoleLogger : IAdvancedLogger
{
    public void Log(string message)
    {
        Console.WriteLine(message);
```

```
    }

    // Can use LogInfo() and LogWarning() as-is
    // Cannot access FormatLog() - it's private to interface
}
```

Static Members in Interfaces (C# 8.0+)

C# 8.0+ allows **static members** in interfaces, including static methods, properties, and fields.[17][18][22][23]

```
public interface ICalculator
{
    // Static field
    static int CalculationCount = 0;

    // Static method with implementation
    static double Add(double a, double b)
    {
        CalculationCount++;
        return a + b;
    }

    // Static property
    static string Version { get; } = "1.0";

    // Instance method
    double Calculate(double x, double y);
}

// Usage - call static members directly on interface
double result = ICalculator.Add(5, 3);
Console.WriteLine(ICalculator.Version);
Console.WriteLine($"Calculations performed: {ICalculator.CalculationCount}");
```

Static Abstract Members (C# 11+):

```
public interface IParsable<TSelf> where TSelf : IParsable<TSelf>
{
    // Static abstract method - must be implemented by type
```

```
    static abstract TSelf Parse(string s);

    // Static virtual method with default implementation
    static virtual bool TryParse(string s, out TSelf result)
    {
        try
        {
            result = TSelf.Parse(s);
            return true;
        }
        catch
        {
            result = default;
            return false;
        }
    }
}
```

Types of Interfaces

1. Normal Interface

Definition: Standard interfaces with multiple method/property declarations that define a contract for behavior.[24][4]

Characteristics:

- Contains multiple members
- Defines complete contract for functionality
- Most commonly used interface type

```
public interface IRepository<T>
{
    void Add(T item);
    void Remove(T item);
    T GetById(int id);
    IEnumerable<T> GetAll();
```

```
    void Update(T item);
    bool Exists(int id);
}

public class UserRepository : IRepository<User>
{
    public void Add(User item) { /* implementation */ }
    public void Remove(User item) { /* implementation */ }
    public User GetById(int id) { /* implementation */ }
    public IEnumerable<User> GetAll() { /* implementation */ }
    public void Update(User item) { /* implementation */ }
    public bool Exists(int id) { /* implementation */ }
}
```

2. Marker Interface (Tagged Interface)

Definition: An **empty interface** with no members, used to **tag** or **mark** classes for special treatment at runtime or compile-time.[25][26][27][24]

Purpose:

- Indicate that a class has certain characteristics
- Used for metadata and reflection
- Enable runtime type checking
- Facilitate design patterns

Common Examples:

- `ISerializable` - Marks class as serializable
- `ICloneable` - Marks class as cloneable
- `IDisposable` - Marks class as needing cleanup

```
// Marker interface - no members
public interface ISerializable
{
    // Empty - just marks the class
}
```

```
public interface ILoggable
{
    // Empty marker interface
}

public class Customer : ISerializable, ILoggable
{
    public string Name { get; set; }
    public string Email { get; set; }
}

// Usage - checking marker interface at runtime
void ProcessObject(object obj)
{
    if (obj is ISerializable)
    {
        Console.WriteLine("Object can be serialized");
        // Special serialization logic
    }

    if (obj is ILoggable)
    {
        Console.WriteLine("Object should be logged");
        // Logging logic
    }
}

Customer customer = new Customer();
ProcessObject(customer);
// Output: Object can be serialized
//         Object should be logged
```

Note: Microsoft's design guidelines recommend using **attributes** instead of marker interfaces in modern C#, but marker interfaces are still easier to check.[27][24]

```
// Modern alternative using attributes
[Serializable]
[Loggable]
public class Customer
{
```

```
    public string Name { get; set; }
}
```

3. Functional Interface

Definition: An interface with a **single method** declaration, similar to functional programming concepts.[27]

Characteristics:

- Contains only one method
- Often used with delegates and lambda expressions
- Represents a single unit of functionality

```
// Functional interface - single method
public interface IComparator<T>
{
    int Compare(T x, T y);
}

public class StringLengthComparator : IComparator<string>
{
    public int Compare(string x, string y)
    {
        return x.Length.CompareTo(y.Length);
    }
}

// Can be used with lambda-like syntax
IComparator<string> comparator = new StringLengthComparator();
int result = comparator.Compare("hello", "world");
```

C# Equivalent: In C#, delegates and `Func<>`, `Action<>` types serve similar purposes:

```
// Instead of functional interface, use delegates
Func<string, string, int> comparator = (x, y) => x.Length.CompareTo(y.Length);
```

Variables in Abstract Classes vs Interfaces

Variables in Abstract Classes

Abstract classes **CAN contain instance variables (fields)** - this is one of the key differences from interfaces.[14][8][7][4]

```
abstract class Animal
{
    // Instance variables (fields)
    protected string name;
    private int age;
    private double weight;

    // Constants
    public const int MaxAge = 100;

    // Static variables
    public static int AnimalCount = 0;

    // Auto-implemented properties
    public string Species { get; set; }

    // Constructor to initialize fields
    protected Animal(string animalName, int animalAge)
    {
        name = animalName;
        age = animalAge;
        AnimalCount++;
    }

    // Abstract method
    public abstract void MakeSound();

    // Concrete method using fields
    public void DisplayInfo()
    {
        Console.WriteLine($"Name: {name}, Age: {age}, Weight: {weight}kg");
    }
}
```

```
class Dog : Animal
{
    // Derived class can have its own fields
    private string breed;

    public Dog(string name, int age, string breed) : base(name, age)
    {
        this.breed = breed;
    }

    public override void MakeSound()
    {
        Console.WriteLine($"{name} the {breed} says: Woof!");
    }
}
```

What Abstract Classes Can Have:

- ☑ Instance fields/variables
- ☑ Static fields
- ☑ Constants
- ☑ Properties (with backing fields)
- ☑ Constructors (to initialize fields)
- ☑ Can store state

Variables in Interfaces

Interfaces **CANNOT contain instance variables or fields** - they can only contain properties, methods, events, and indexers.[28][4][14]

Pre-C# 8.0:

```
public interface IVehicle
{
    // X NOT ALLOWED - cannot have fields
    // private string name;
```

```
    // private int speed;

    // ☑ ALLOWED - property declarations (no backing field in interface)
    string Brand { get; set; }
    int MaxSpeed { get; }

    // ☑ ALLOWED - constants
    const int MinSpeed = 0;

    // Method declarations
    void Start();
    void Stop();
}
```

C# 8.0+:

```
public interface IVehicle
{
    // X STILL NOT ALLOWED - cannot have instance fields
    // private string name;

    // ☑ ALLOWED - static fields (C# 8.0+)
    static int VehicleCount = 0;

    // ☑ ALLOWED - constants
    const int MinSpeed = 0;

    // ☑ ALLOWED - properties with default implementation
    string Brand { get; set; }

    // ☑ ALLOWED - default method using properties
    void DisplayInfo()
    {
        Console.WriteLine($"Brand: {Brand}");
        Console.WriteLine($"Total vehicles: {VehicleCount}");
    }
}
```

Comparison: State Management

Feature	Abstract Class	Interface
Instance fields/variables	☑ Yes	❌ No
Can store state	☑ Yes	❌ No (no instance state)
Static fields	☑ Yes	☑ Yes (C# 8.0+)
Constants	☑ Yes	☑ Yes
Properties with backing fields	☑ Yes	❌ No (properties only)
Constructors	☑ Yes	❌ No
Initialize state	☑ Yes (in constructor)	❌ No

Abstract Class vs Interface - Comprehensive Comparison

Feature	Abstract Class	Interface (Pre-C# 8.0)	Interface (C# 8.0+)
Instantiation	❌ Cannot instantiate	❌ Cannot instantiate	❌ Cannot instantiate
Multiple Inheritance	❌ No (single inheritance)	☑ Yes	☑ Yes
Fields/Variables	☑ Yes	❌ No	❌ No (only static)
Constructors	☑ Yes	❌ No	❌ No
Destructors	☑ Yes	❌ No	❌ No
Abstract Methods	☑ Yes	☑ Yes (all by default)	☑ Yes
Concrete Methods	☑ Yes	❌ No	☑ Yes (default methods)
Static Methods	☑ Yes	❌ No	☑ Yes
Access Modifiers	☑ All (public, private, protected, internal)	❌ No (all public)	☑ Yes (all modifiers)
Default Implementation	☑ Yes	❌ No	☑ Yes

Virtual Methods	☑ Yes	❌ No	☑ Yes
Can Store State	☑ Yes	❌ No	❌ No
Properties	☑ Yes (with backing fields)	☑ Yes (declaration)	☑ Yes (with default)
Events	☑ Yes	☑ Yes (declaration)	☑ Yes (with default)
Indexers	☑ Yes	☑ Yes (declaration)	☑ Yes (with default)
Versioning	☑ Easy (add members)	❌ Hard (breaks code)	☑ Easy (default methods)
Performance	Slightly faster	Slightly slower	Slightly slower

When to Use Abstract Classes vs Interfaces

Use Abstract Classes When:

1. **Related classes share common behavior and state**

```
abstract class Employee
{
    protected string name;  // Shared state
    protected decimal salary;

    public abstract decimal CalculateBonus();

    public void DisplayInfo()  // Shared behavior
    {
        Console.WriteLine($"{name}: ${salary}");
    }
}
```

2. **You need constructors to initialize state**
3. **You want to provide default implementations**
4. **Classes are closely related (IS-A relationship)**

5. **You need to use non-public members (protected, private)**
6. **You want to evolve base class functionality over versions**

Use Interfaces When:

1. **Unrelated classes need to share behavior (CAN-DO relationship)**

```
interface IFlyable
{
    void Fly();
}

class Bird : IFlyable { /* ... */ }
class Airplane : IFlyable { /* ... */ }
class Superman : IFlyable { /* ... */ }
```

2. **You need multiple inheritance**
3. **You want to define a contract without implementation**
4. **Classes don't share common state**
5. **You want plug-in architecture**
6. **You need to support cross-cutting concerns**

Combining Both:

```
// Abstract class for base identity and state
abstract class Employee
{
    protected string name;
    protected decimal salary;

    public Employee(string name, decimal salary)
    {
        this.name = name;
        this.salary = salary;
    }
```

```
    public abstract decimal CalculateBonus();
}

// Interfaces for capabilities
interface ITrainable
{
    void Train(string course);
}

interface IPromotable
{
    void Promote(string newPosition);
}

// Combine both
class Manager : Employee, ITrainable, IPromotable
{
    public Manager(string name, decimal salary) : base(name, salary) { }

    public override decimal CalculateBonus()
    {
        return salary * 0.15m;
    }

    public void Train(string course)
    {
        Console.WriteLine($"{name} is training in {course}");
    }

    public void Promote(string newPosition)
    {
        Console.WriteLine($"{name} promoted to {newPosition}");
    }
}
```

How Abstraction is Achieved in C#

Level 1: Using Abstract Classes (Partial Abstraction)

Abstract classes provide **0-100% abstraction** - can have both abstract and concrete members.[6][7][3]

```
// Partial abstraction example
abstract class PaymentProcessor
{
    // Concrete implementation - shared logic
    public void LogPayment(decimal amount)
    {
        Console.WriteLine($"Payment of ${amount} logged at {DateTime.Now}");
    }

    // Abstract - each processor implements differently
    public abstract bool ProcessPayment(decimal amount);
    public abstract void RefundPayment(string transactionId);
}

class CreditCardProcessor : PaymentProcessor
{
    public override bool ProcessPayment(decimal amount)
    {
        Console.WriteLine($"Processing ${amount} via Credit Card");
        return true;
    }

    public override void RefundPayment(string transactionId)
    {
        Console.WriteLine($"Refunding transaction {transactionId}");
    }
}
```

Achieves Abstraction By:

- Hiding credit card processing details
- Exposing only ProcessPayment() and RefundPayment()
- Providing shared logging functionality

Level 2: Using Interfaces (Complete Abstraction)

Interfaces provide **100% abstraction** (traditionally) - only declarations, no implementation.[3][4]

```
// Complete abstraction example
public interface IDataService
{
    void Save(object data);
    object Load(int id);
    void Delete(int id);
}

public class DatabaseService : IDataService
{
    public void Save(object data)
    {
        // SQL database implementation hidden
        Console.WriteLine("Saving to SQL database");
    }

    public object Load(int id)
    {
        Console.WriteLine("Loading from SQL database");
        return new object();
    }

    public void Delete(int id)
    {
        Console.WriteLine("Deleting from SQL database");
    }
}

public class FileService : IDataService
{
    public void Save(object data)
    {
        // File system implementation hidden
        Console.WriteLine("Saving to file system");
    }

    public object Load(int id)
    {
```

```
        Console.WriteLine("Loading from file system");
        return new object();
    }

    public void Delete(int id)
    {
        Console.WriteLine("Deleting from file system");
    }
}

// Client code doesn't know implementation details
void ProcessData(IDataService service, object data)
{
    service.Save(data);  // Abstraction - don't care how it saves
}
```

Achieves Abstraction By:

- Hiding all implementation details
- Defining only contract (what, not how)
- Allowing different implementations behind same interface

Level 3: C# 8.0+ Default Methods (Flexible Abstraction)

```
public interface ILogger
{
    // Abstract - must implement
    void Log(string message);

    // Default implementation - optional to override
    void LogWithTimestamp(string message)
    {
        Log($"[{DateTime.Now}] {message}");
    }

    // Private helper - hidden from implementers
    private string FormatMessage(string level, string message)
    {
        return $"[{level}] {message}";
```

```
    }

    // Public default using private helper
    public void LogError(string message)
    {
        Log(FormatMessage("ERROR", message));
    }
}
```

Achieves Abstraction By:

- Hiding implementation of LogWithTimestamp and LogError
- Hiding helper method FormatMessage completely
- Exposing only essential Log() method for implementation

Purpose of Different Methods in Interfaces

1. Abstract Methods (Default - Must Implement)

Purpose: Define mandatory contract that all implementers must provide.[29][12][7]

```
public interface IShape
{
    // Abstract methods - no implementation
    double CalculateArea();
    double CalculatePerimeter();
}
```

When to Use:

- Core functionality that varies per implementation
- Must be implemented by every class
- No sensible default exists

2. Default Methods (C# 8.0+ - Optional Implementation)

Purpose: Provide reusable functionality that implementers can use as-is or override.[19][15][12][13]

```
public interface IShape
{
    double CalculateArea();

    // Default method - optional to override
    void Display()
    {
        Console.WriteLine($"Area: {CalculateArea()} square units");
    }

    // Default method using other interface members
    bool IsLargerThan(IShape other)
    {
        return this.CalculateArea() > other.CalculateArea();
    }
}
```

When to Use:

- Common functionality across implementations
- Backward compatibility when extending interfaces
- Utility methods that don't need customization
- Versioning scenarios

3. Virtual Methods (C# 8.0+ - Can Override)

Purpose: Provide default implementation that can be overridden.[21][29][13]

```
public interface ILogger
{
    void Log(string message);

    // Virtual - can be overridden
    public virtual void LogWarning(string message)
    {
        Log($"WARNING: {message}");
    }
```

```
}

class CustomLogger : ILogger
{
    public void Log(string message)
    {
        Console.WriteLine(message);
    }

    // Override virtual method
    public void LogWarning(string message)
    {
        Console.WriteLine($"⚠ {message}");
    }
}
```

When to Use:

- Default behavior that some implementations may want to customize
- Provide common implementation but allow flexibility

4. Private Methods (C# 8.0+ - Internal Helpers)

Purpose: Helper methods used by other default methods, hidden from implementers.[20][21][13]

```
public interface ICalculator
{
    double Calculate(double x, double y);

    // Private helper - not accessible to implementers
    private string FormatResult(double result)
    {
        return $"Result: {result:F2}";
    }

    // Public default using private helper
    public void DisplayResult(double x, double y)
    {
        double result = Calculate(x, y);
        Console.WriteLine(FormatResult(result));
```

```
    }
}
```

When to Use:

- Break down complex default methods
- Reuse logic within interface
- Keep implementation details hidden

5. Protected Methods (C# 8.0+ - For Derived Interfaces)

Purpose: Available in derived interfaces, not accessible to implementing classes.[13][20]

```
public interface IBaseLogger
{
    void Log(string message);

    // Protected - available in derived interfaces
    protected void LogInternal(string level, string message)
    {
        Log($"[{level}] {message}");
    }
}

public interface IAdvancedLogger : IBaseLogger
{
    // Can use protected method from base
    void LogDebug(string message)
    {
        LogInternal("DEBUG", message);
    }
}
```

When to Use:

- Interface hierarchies
- Share functionality between related interfaces
- Provide utilities for derived interfaces

6. Static Methods (C# 8.0+ - Utility Methods)

Purpose: Provide utility functionality without needing an instance.[18][22][17][21]

```
public interface IMathOperations
{
    double Calculate(double x, double y);

    // Static method - utility function
    static double Square(double x)
    {
        return x * x;
    }

    // Static method with logic
    static double Average(params double[] numbers)
    {
        return numbers.Sum() / numbers.Length;
    }
}

// Usage - call directly on interface
double squared = IMathOperations.Square(5);
double avg = IMathOperations.Average(1, 2, 3, 4, 5);
```

When to Use:

- Utility functions related to interface
- Factory methods
- Helper functions that don't need instance state
- Parsing or validation logic

Summary: Quick Reference Guide

Abstract Class Quick Reference

☑ **Use For**:

- Sharing state (fields) across related classes
- Providing partial implementation
- Establishing IS-A relationships
- Requiring constructors for initialization

💡 **Key Features**:

- Can have fields, constructors, destructors
- Can mix abstract and concrete members
- Single inheritance only
- All access modifiers supported

Interface Quick Reference (C# 8.0+)

☑ **Use For**:

- Defining contracts without state
- Supporting multiple inheritance
- Establishing CAN-DO relationships
- Creating plug-in architectures

💡 **Key Features**:

- No instance fields (only properties, static fields)
- Can have default implementations
- Multiple interface inheritance
- Access modifiers in default methods

Types of Interfaces

Type	Members	Purpose	Example Use
Normal	Multiple methods/properties	Define complete contract	IRepository, ILogger
Marker	Empty (no members)	Tag classes for runtime checks	ISerializable, ILoggable
Functional	Single method	Single responsibility	IComparator, IValidator

Method Types in Interfaces (C# 8.0+)

Method Type	Access	Implementation	Override
Abstract (default)	public	No	Must implement
Default	public	Yes	Optional
Virtual	public	Yes	Can override
Private	private	Yes	Cannot access
Protected	protected	Yes	Derived interfaces only
Static	public	Yes	Not overridable

This comprehensive guide covers all aspects of abstraction in C#, including abstract classes, interfaces, their differences, types of interfaces, variables, methods (including C# 8.0+ features), and practical examples for achieving abstraction.[15][12][7][1][17][18][21][29][4][20][3][13]

**

1. https://www.c-sharpcorner.com/article/c-sharp-abstract-class-with-examples/
2. https://www.w3schools.com/cs/cs_abstract.php
3. https://www.geeksforgeeks.org/c-sharp/difference-between-abstract-class-and-interface-in-c-sharp/
4. https://www.linkedin.com/pulse/understanding-difference-between-abstract-class-interface-kamal-oidjf
5. https://www.shiksha.com/online-courses/articles/difference-between-abstract-class-and-interface-in-c-blogId-151941
6. https://www.programiz.com/csharp-programming/abstract-class
7. https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/abstract
8. https://www.scholarhat.com/tutorial/csharp/a-deep-dive-into-csharp-abstract-class
9. https://www.c-sharpcorner.com/UploadFile/955025/C-Sharpinterviewquestionpart7can-an-abstract-class-have-a-constr/

10. https://stackoverflow.com/questions/5601777/constructor-of-an-abstract-class-in-c-sharp
11. https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/how-to-define-abstract-properties
12. https://www.geeksforgeeks.org/c-sharp/default-interface-methods-in-c-sharp/
13. https://www.c-sharpcorner.com/article/interfaces-in-c-sharp8-0/
14. https://byjus.com/gate/difference-between-abstract-class-and-interface-in-c-sharp/
15. https://dev.to/dotnetfullstackdev/default-interface-methods-and-properties-implementation-in-c-80-3ean
16. https://learn.microsoft.com/en-us/dotnet/csharp/advanced-topics/interface-implementation/default-interface-methods-versions
17. https://stackoverflow.com/questions/9415257/how-can-i-implement-static-methods-on-an-interface
18. https://blog.jetbrains.com/dotnet/2023/03/14/static-interface-members-generic-attributes-auto-default-structs-using-csharp-11-in-rider-and-resharper/
19. https://www.c-sharpcorner.com/article/default-interface-methods/
20. https://jeremybytes.blogspot.com/2019/11/c-8-interfaces-public-private-and.html
21. https://www.c-sharpcorner.com/article/method-implementation-private-static-members-in-c-sharp-interfaces-new-feature-i/
22. https://jeremybytes.blogspot.com/2019/12/c-8-interfaces-static-members.html
23. https://michalbrylka.github.io/posts/static-interface-members/
24. https://www.usenix.org/system/files/login/articles/1136-mccluskey.pdf
25. https://www.tutorialspoint.com/what-is-a-marker-or-tagged-interface-in-java
26. https://blog.ndepend.com/marker-interface-isnt-pattern-good-idea/
27. https://stackoverflow.com/questions/1023068/what-is-the-purpose-of-a-marker-interface
28. https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/types/interfaces
29. https://www.tutorialsteacher.com/csharp/csharp-interface
30. https://dotnettutorials.net/course/csharp-dot-net-tutorials/
31. https://www.geeksforgeeks.org/c-sharp/csharp-programming-language/

32. https://music.youtube.com/playlist?list=PLmQ-r_fmIh3JkW7PiPrD_PpIvpeYOFohN

33. https://www.youtube.com/playlist?list=PLPZvv4Sjz6uEMR4OV9dCsn8gH6N3x-Igj

34. https://www.youtube.com/playlist?list=PLQVCSt-IuOa2Lr5IUVIptZOCKGky7oHLL

35. https://www.youtube.com/playlist?list=PL8o_tiXLMw5eSOX2Lr1lrA794kp-leyHL

36. https://www.youtube.com/playlist?list=PLJhxHp6j1l-9RHSkg_CZXZAI45aVmrQXD

37. https://www.youtube.com/playlist?list=PLRBYRRt126hoGSx4qtp7ePe-9FshnLBdO

38. https://www.youtube.com/playlist?list=PL_talnLgOSYVe90Yczm8JSmysQV3rlwql

39. https://www.youtube.com/playlist?list=PLYBaPauECQ5oPNLS1lapSDmTYuw-McVUf

40. https://en.wikipedia.org/wiki/Marker_interface_pattern

41. https://www.reddit.com/r/csharp/comments/1c3m5nd/understanding_c_8_default_interface_methods/

42. https://learn.microsoft.com/en-us/archive/msdn-magazine/2009/october/functional-programming-for-everyday-net-development

43. https://stackoverflow.com/questions/53278587/do-c-sharp-8-default-interface-implementations-allow-for-multiple-inheritance

44. https://www.geeksforgeeks.org/c-sharp/c-sharp-abstract-classes/

45. https://www.youtube.com/playlist?list=PLcx-MjNucvBrz6PU8LtyE3wKZ09-R5tu_

46. https://www.youtube.com/playlist?list=PLv3i3aAkZiZI_ZvH5Jbi6Byt9EE5XCnqM

47. https://www.youtube.com/playlist?list=PL-RaxaVBEGBr2y2S0DK_XC9lkFh-OuA5c

48. https://www.youtube.com/playlist?list=PL_pbwdIyffsnH3XJb66FDIHh1yHwWC26I

49. https://www.youtube.com/playlist?list=PLo80fWiInSIPRB6O3O1H38sWg5mbsexIZ

50. https://www.youtube.com/playlist?list=PL5fOKT7XR42Ph5iSKpDADlNW03t-SnbB_

51. https://www.youtube.com/playlist?list=PLxOWTEJLkn6rPHhoDfeTT4moLJo1oT9mD

52. https://www.youtube.com/playlist?list=PLQNxlCDSRO9PIWFoyhl8WJjV4g5r6MbQx

53. https://www.youtube.com/playlist?list=PLRKyZvuMYSIOZJo7ytuGsl0Sw3JyDHL6W

54. https://www.youtube.com/playlist?list=PL4jg6txzaSpf32hRt7kKyGq_4yZLoxMEC

55. https://www.youtube.com/playlist?list=PLqrE-6VqzcGYzngqd5eaBgIp9_I5AcVd1

56. https://www.youtube.com/playlist?list=PLVs9OVYj7CFcwqQfweZd2tgSlU3_yixhF

57. https://www.youtube.com/playlist?list=PLv7qaVoFdyLHBweMU23zyqhcKGUPS93Ru

58. https://www.youtube.com/playlist?list=PL0YTS3lJHMdoh5pTHb9x7RuUhu2U97MZq

59. https://www.youtube.com/playlist?list=PLqrcj3pm68R0i5oBjMrE5sh62FuBELfMZ

60. https://www.youtube.com/playlist?list=PL6fIXCB2Mcv--z714Je5NrdRXy_S3OCs9

61. https://ironpdf.com/blog/net-help/csharp-protected/

62. https://stackoverflow.com/questions/516148/why-cant-i-have-protected-interface-members

63. https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/static-classes-and-static-class-members

64. https://www.w3schools.com/cs/cs_access_modifiers.php

65. https://iqratechnology.com/academy/c-sharp-training/c-abstract/

66. https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/access-modifiers

67. https://www.reddit.com/r/csharp/comments/1e2cccd/whats_the_purpose_of_having_private_methods_in/